PROJECT GUIDE | 28 OCTOBER 2026

E-Commerce Customer Behavior Analytics

A practical guide to the application, its Olist dataset integration, analytics workflows, personalized recommendations, and local setup.

PROJECT AT A GLANCE	CURRENT IMPLEMENTATION
Web application	React + TypeScript frontend, FastAPI backend, PostgreSQL database
Active data option	Olist Brazilian marketplace dataset, orders from 2016-2018; values shown in BRL
Alternative data option	UCI Online Retail, UK retailer transactions from 2010-2011; values shown in GBP
Core capabilities	Dashboard, customer/RFM analysis, segmentation, basket rules, forecasts, recommendations, run history

What the project does

This project turns imported retail transactions into a local analytics application. Users explore sales and customer summaries, run batch analytics, inspect saved results, and view product suggestions for a selected customer. The application separates its user interface, API, database, and analytics code so each part has a clear role.

The project was first built around the UCI Online Retail data model. Olist support now allows the same dashboard and batch analytics to read the Brazilian marketplace tables through a set of PostgreSQL compatibility views. Recommendations query the Olist tables directly.

Current status

You reported that the application is working after the Olist setup and recommendations integration. This guide describes the implemented project and its operating assumptions; it does not claim a fresh independent test or build in this documentation run.

Data age matters

Olist is richer in orders, products, sellers, payments, reviews, and delivery data than the original retail workbook, but it is historical (2016-2018), not a live feed and not a current 2026 market sample. Treat trends as patterns in that period.

1. Architecture and data

The browser calls the API over HTTP/JSON. FastAPI validates inputs and reads or writes PostgreSQL via SQLAlchemy. Long-running analytics are stored as runs; the frontend polls their status and then displays the saved result.

LAYER	TOOLS / RESPONSIBILITY
Frontend	React, TypeScript, Vite; pages, forms, tables, charts, API client
Backend	Python, FastAPI, Pydantic, SQLAlchemy; REST endpoints, validation, database access
Analytics	Pandas / NumPy / scikit-learn / mlxtend; cleaning, RFM, clustering, association rules, forecasting
Database	PostgreSQL; source records, normalized analytics tables, saved run results
Local development	Vite frontend on port 5173; API on port 8000; PostgreSQL typically on port 5432

Data sources and database shape

The project can use one configured dataset/database at a time. Select the database by setting DATABASE_URL in backend/.env and restarting the API. Never put real database credentials in the frontend or commit .env files.

SOURCE	WHAT IS IN IT	HOW THE APP USES IT
Olist	Nine public CSVs: customers, geolocation, order items, payments, reviews, orders, products, sellers, category translations.	Directly powers the recommendation page; compatibility views adapt key order/customer/product/item fields for the existing analytics.
UCI Online Retail	Transaction workbook with invoice, stock code, description, quantity, price, date, customer and country.	Original import and analytics path; monetary values remain GBP.

Olist database objects

The nine imported olist_* and translation tables preserve the source dataset. The Olist compatibility SQL adds views customers, products, orders and order_items, plus analytics_runs, customer_segments, association_rules, forecasts and analytics_customer_map. The views adapt names and types; they do not replace the imported source tables.

Important assumptions in the adapter: analytics use delivered Olist orders for customer purchase behavior; each Olist order item is represented as quantity 1 because the source item table has no quantity column; item price is used as item revenue, and currency is BRL. Customer IDs in compatibility views are stable integer keys mapped from Olist customer_unique_id.

2. Analytics and user workflows

The standard workflow is: load data, check the detected source and readiness, run the analysis from its page, then review results and run history. Batch runs are asynchronous and persisted in PostgreSQL.

Dashboard

Shows overall sales, order and customer metrics, average order value, repeat purchase rate and cancellation measures, plus time trends, top products, and geographic breakdowns. Amount formatting follows the selected source currency (BRL for Olist, GBP for UCI).

Customers and RFM

RFM measures Recency (days since last purchase), Frequency (distinct orders), and Monetary (spend). Scores are assigned by quintiles; recency scoring is reversed so recent customers receive higher scores. Rule-based segments include Champions, Loyal Customers, Potential Loyalists, New Customers, Need Attention, About to Sleep, At Risk, Can't Lose Them, and Hibernating.

Customer segmentation

K-Means clusters customers using recency, frequency, and monetary values. The implementation log-transforms skewed values, standardizes features, evaluates candidate cluster counts using silhouette score, and saves cluster profiles and a PCA projection for visualization. The candidate range is k=2 through k=8, bounded by the number of customers.

Products and baskets

Apriori mines products that occur together in an order and produces association rules. Support is the share of baskets containing an itemset; confidence estimates P(B given A); lift compares that confidence with the baseline frequency of B. For bounded memory use, the analysis limits input to up to 150 frequent products and itemsets of length at most 3.

Forecasting

Forecasting aggregates revenue by day and compares a Gradient Boosting model with a seasonal-naive baseline. It holds out the final 28 days for evaluation, uses revenue lags (28, 35, and 42 days) plus shifted rolling/calendar features, reports MAE and RMSE, then refits on available history for a forecast horizon up to 28 days. These are historical-data forecasts, not guarantees.

Run history

Analytics runs have a type and status (pending, running, completed, or failed), parameters, timestamps, duration, and saved results. The Run History page makes previous work and failures inspectable.

3. Personalized recommendations

The Recommendations page asks for a customer, shows their delivered purchase history, and suggests products they have not bought. It supports both the legacy UCI schema and Olist. Olist suggestions display product IDs and translated category names because the public catalog does not provide human-readable product titles.

How it ranks suggestions

- Builds a set of delivered products for the selected customer and for other customers.
- Measures product-history overlap with Jaccard similarity: shared products divided by the union of the two product sets.
- Considers up to 100 nearest overlapping customers; unseen products are scored using the sum of neighbor similarities.
- If peer-based candidates are unavailable, falls back to popular delivered products that the selected customer has not purchased.
- Explains peer-based recommendations with the number of similar customers that purchased the product; fallback recommendations are marked as popular items.

Olist setup and data loading

In pgAdmin, connect to the Olist database, create the nine source tables, and import the nine CSVs in the foreign-key order in the Olist README. Then run the compatibility script after all source data is loaded. Point backend/.env at that database and restart FastAPI. The UI should identify Olist and BRL; analytics pages require the compatibility objects to exist.

STEP	ACTION / LOCATION
1	Extract the Olist archive; retain all nine CSV files.
2	Run Databas_Olist/02_create_tables.sql on the selected Olist database.
3	Import customers, sellers, products, category translation, geolocation, orders, order items, payments, and reviews in the documented dependency order.
4	Run Databas_Olist/03_enable_project_analytics.sql after imports. It creates analytics views and saved-result tables.
5	Set backend/.env DATABASE_URL to your PostgreSQL host, port, database, username and password. Restart the API.
6	Open /api/data-source to inspect source/readiness, then run RFM, clustering, basket, or forecasting from the corresponding UI page.

If a dashboard page reports analytics are not ready, check that the API is connected to the intended database, all CSV imports completed, and SQL script 03 ran in that same database. The recommendation page may work directly from Olist source tables even if the compatibility setup is incomplete.

4. Run locally, API, and limitations

Start PostgreSQL first. Use separate PowerShell terminals for the backend and frontend. Run backend commands from the backend folder and frontend commands from frontend.

Backend

```
cd C:\Sahil\sem7\BDA\Project\backend
.\.venv\Scripts\Activate.ps1
uvicorn app.main:app --reload --port 8000
```

If your active virtual environment is named myenv and is already activated, omit the activation command. Install backend requirements into the environment you actually use. The backend expects DATABASE_URL in backend/.env.

Frontend

```
cd C:\Sahil\sem7\BDA\Project\frontend
npm install
npm run dev
```

Useful local pages and API routes

PURPOSE	ADDRESS
Frontend	http://localhost:5173
API health (database connection check)	http://127.0.0.1:8000/api/health
Detected data source / analytics readiness	http://127.0.0.1:8000/api/data-source
Interactive API documentation	http://127.0.0.1:8000/docs
Analytics	/api/dashboard/summary, /api/customers, /api/rfm/summary, /api/clusters/latest, /api/association-rules, /api/forecast/latest
Recommendations and runs	/api/recommendations/customers, /api/recommendations/{customer_id}, /api/runs

Project pages

Dashboard; Customers & RFM; Segmentation; Products & Baskets; Recommendations; Forecasting; Data Import & Quality; Run History. The workbook upload/import workflow is UCI-specific; Olist CSVs are imported to their source tables with pgAdmin.

Scope and limitations

- Olist records cover 2016-2018 and represent Brazilian marketplace activity; they cannot establish current Indian demand or 2026 trends.
- The compatibility layer intentionally presents a simplified shared schema; some source details (payments, reviews, seller geography) are not included in the current dashboard analytics.
- Olist geolocation is retained as a lookup table and is not a full geospatial analysis feature.
- Recommendations are collaborative filtering from historical purchases, not a trained ranking model and not a causal measure of sales impact.
- Batch jobs run in the API process, suitable for local coursework; production deployments should use a durable job queue and stronger operational controls.

Project files consulted

README.md; backend/app/api.py; backend/app/services/analytics.py; backend/app/services/recommendations.py; Databas_Olist/README.md; Databas_Olist/03_enable_project_analytics.sql.